

به نام خدا



دانشگاه تهران
دانشکده مهندسی
برق و کامپیوتر



پروژه ی درس درستی سنجی مدل های HDL

مهسا ابرغانی اقدم - ۸۱۰۱۰۳۰۵۳

□ مرحله چهارم: طراحی واحدهای Checker و Scoreboard

شرح: در این قسمت باید آخرین قسمت پازل بستر درستی یابی را طراحی نمایید. در تمرینات پیشین، قسمت‌هایی طراحی نمودید که به واسطه آن توانستید عملکرد مدار DUV را شبیه‌سازی نمایید (Golden Model)، قسمتی را طراحی نمودید تا بتوانید ورودی‌های مورد نیاز خود را به ساختاری که طراحی نموده‌اید، ارسال نمایید (Stimuli). تاکنون همه قسمت‌های مهم را طراحی نموده‌اید. در این تمرین، آخرین قسمت پازل را باید طراحی نمایید. هدف از این تمرین، طراحی قسمتی است که بتواند ایرادات کشف شده در سیستم را آشکار نموده و تشخیص دهد. طبیعی است که این تشخیص باید با مقایسه خروجی‌های تولید شده از دو قسمت Golden Model و DUV انجام شود. شما باید خروجی گلدن مدل را با خروجی دیزاین مقایسه کنید از نظر زمان و خروجی.

Checker:

```
module Checker (  
    input Logic      clk,  
    input Logic      reset,  
    input Logic [31:0] golden_HRDATA,  
    input Logic [31:0] tb_HRDATA,  
    input Logic      golden_HREADY,  
    input Logic      tb_HREADY,  
    input Logic [31:0] address,  
    output Logic [15:0] error_count  
);
```

```

integer filecheck;

initial begin
    filecheck = $fopen("checker_output.txt", "w");
    if (filecheck == 0) begin
        $display("Error: Could not open file checker_output.txt");
        $finish;
    end
end

logic [15:0] local_error_count;

always_ff @(posedge clk or posedge reset) begin
    if (reset) begin
        local_error_count <= 0;
    end else begin
        if (golden_HREADY && tb_HREADY) begin
            if (golden_HRDATA != tb_HRDATA) begin
                local_error_count <= local_error_count + 1;
                $fwrite(filecheck, "Error: Time %0t Address %h Golden %h DUV
%h\n", $time, address, golden_HRDATA, tb_HRDATA);
            end else if (golden_HRDATA == tb_HRDATA) begin
                $fwrite(filecheck, "OK: Time %0t Address %h Golden %h DUV
%h\n", $time, address, golden_HRDATA, tb_HRDATA);
            end
        else begin
            $fwrite(filecheck, "Not Ready: Time %0t Address %h Golden %h DUV
%h\n", $time, address, golden_HRDATA, tb_HRDATA);
        end
    end
end

assign error_count = local_error_count;

final begin
    $fclose(filecheck);
end
endmodule

```

توضیح:

در ابتدای شبیه‌سازی، فایل checker_output.txt برای ثبت گزارش‌ها باز می‌شود. اگر باز کردن فایل با خطا مواجه شود، پیام

خطا نمایش داده شده و شبیه‌سازی متوقف می‌شود. این فایل به‌عنوان محلی برای ذخیره نتایج مقایسه بین داده‌ها مورد استفاده قرار می‌گیرد.

در هر سیکل کلاک، اگر هر دو ماژول مدل مرجع و طراحی تحت تست آماده باشند، داده‌های خروجی آن‌ها با یکدیگر مقایسه می‌شوند. این بررسی فقط زمانی انجام می‌شود که سیگنال‌های golden_HREADY و tb_HREADY فعال باشند و نشان دهند که داده‌ها معتبر هستند.

اگر داده‌های خروجی مدل مرجع golden_HRDATA و طراحی تحت تست tb_HRDATA با یکدیگر متفاوت باشند، شمارنده خطا افزایش می‌یابد و یک پیام خطا شامل زمان، آدرس و مقدار داده‌ها در فایل گزارش ثبت می‌شود. در صورتی که داده‌ها برابر باشند، یک پیام OK به همراه اطلاعات مشابه در فایل ذخیره می‌شود.

اگر یکی از ماژول‌ها آماده نباشد، یعنی سیگنال‌های golden_HREADY یا tb_HREADY غیرفعال باشند، یک پیام Not Ready به همراه اطلاعات زمان، آدرس و داده‌ها در فایل ثبت می‌شود تا نشان دهد که مقایسه‌ای انجام نشده است.

شمارنده خطا که تعداد خطاهای شناسایی شده را ثبت می‌کند، به خروجی ماژول error_count متصل است تا در خارج از ماژول قابل مشاهده باشد. در پایان شبیه‌سازی، فایل گزارش به‌درستی بسته می‌شود تا اطمینان حاصل شود که تمامی اطلاعات ذخیره شده‌اند.

خروجی:

Not Ready: Time 105 Address 00000000 Golden xxxxxxxx DUV xxxxxxxx

Not Ready: Time 115 Address 00000000 Golden xxxxxxxx DUV xxxxxxxx

Not Ready: Time 125 Address 00000000 Golden xxxxxxxx DUV xxxxxxxx

OK: Time 245 Address 00000002 Golden 9b75ac36 DUV 9b75ac36

OK: Time 385 Address 00000001 Golden 22568d44 DUV 22568d44

Error: Time 735 Address 00000001 Golden 5c8e6fb9 DUV 7163b1e2

طبق سناریو های رندومی که نوشتیم و خروجی های دیزاین و مدل طلایی که به چند دادیم و با توجه به fsm سیستم که خانم محمدی برامون رسم کرده بود توی سناریو توسط چکر یک تفاوت در خروجی ها رو کشف کرده، که به خاطر تفاوت زمانبندی دیزاین با مدل طلاییه و برای اصلاح مشکل نیز باید کد اصلی تغییر کنه

OK: Time 905 Address 00000000 Golden 55fb75ab DUV 55fb75ab

OK: Time 965 Address 00000000 Golden 09d79913 DUV 09d79913

Not Ready: Time 1055 Address 00000000 Golden xxxxxxxx DUV 55fb75ab

Not Ready: Time 1165 Address 00000002 Golden xxxxxxxx DUV 09d79913

Error: Time 1225 Address 00000002 Golden 1ac90d35 DUV 09d79913

Error: Time 1285 Address 00000002 Golden 1ac90d35 DUV 09d79913

Error: Time 1345 Address 00000002 Golden 1ac90d35 DUV 09d79913

Error: Time 1405 Address 00000002 Golden 1ac90d35 DUV 09d79913

Error: Time 1465 Address 00000002 Golden 1ac90d35 DUV 09d79913

Error: Time 1525 Address 00000002 Golden 1ac90d35 DUV 09d79913

Error: Time 1585 Address 00000002 Golden 1ac90d35 DUV 09d79913

طبق سناریو های رندومی که نوشتیم و خروجی های دیزاین و مدل طلایی که به چند دادیم و با توجه به fsm سیستم که خانم محمدی برامون رسم کرده بود توی سناریو ها توسط چکر یک تفاوت در خروجی ها رو کشف کرده، که به خاطر تفاوت زمانبندی دیزاین با مدل طلاییه و برای اصلاح مشکل نیز باید کد اصلی تغییر کنه

Scoreboard:

Scoreboard

- ▶ Temporary holding location for information that the checker will require
- ▶ Checker uses scoreboard in two ways:
- ▶ First method:
 - Checker contains the reference model
 - Scoreboard examines the inputs for transactions to occur and store the information for later use
 - When the checker observes some condition on the outputs of the DUV, it makes a call to the scoreboard to get the data (*callback*).
 - Scoreboard implementation follows checker's

Scoreboard

- ▶ Second method:
 - Scoreboard is the reference model and does the expected result calculation based on the input stimulus it observes
 - When the checker observes DUV output events, it queries the scoreboard for the expected data and performs the compare
- ▶ Scoreboard and stimulus generator must be independent for component re-use at later levels of the verification hierarchy
- ▶ The information about the commands that are outstanding in the DUV is stored in a table. The index into the table is the command tag

```
module sdram_scoreboard (  
    input Logic      clk_in,  
    input Logic      rst_in,  
    input Logic      wr_enable_in,  
    input Logic      sel_in,  
    input Logic [31:0] addr_in,  
    input Logic [31:0] wr_data_in,  
    input Logic      ready_out,  
    input Logic [31:0] rd_data_out,  
  
    output Logic [31:0] total_ops,  
    output Logic [31:0] read_ops,  
    output Logic [31:0] write_ops,  
    output Logic [31:0] invalid_ops  
);  
    integer log_file;  
    initial begin  
        log_file = $fopen("monitor_log.txt","w");  
        invalid_ops = 0;
```

```

        write_ops = 0;
        read_ops = 0;
    end

    always @(posedge clk_in or posedge rst_in) begin
        if (rst_in) begin
            read_ops = read_ops;
            write_ops = write_ops;
            write_ops = write_ops;
            invalid_ops = invalid_ops;
        end else
            if (sel_in && wr_enable_in) begin
                write_ops = write_ops + 1;
                $fwrite(log_file,"WRITE_OP: | Time : %0t | HWDATA : %h |
HADDR: %h\n", $time , wr_data_in , addr_in);
            end else if (sel_in && !wr_enable_in) begin
                read_ops = read_ops + 1;
                $fwrite(log_file,"READ_OP: | Time : %0t | HRDATA : %h |
HADDR: %h\n", $time , rd_data_out , addr_in);
            end else if (ready_out && !sel_in && wr_enable_in) begin
                invalid_ops = invalid_ops + 1;
                $fwrite(log_file,"INVALID_OP: | Time : %0t | HRDATA : %h
| HWDATA: %h | HADDR: %h\n", $time , rd_data_out , wr_data_in , addr_in);
            end else if (ready_out && !sel_in && !wr_enable_in) begin
                invalid_ops = invalid_ops + 1;
                $fwrite(log_file,"INVALID_OP: | Time : %0t | HRDATA : %h |
HWDATA: %h | HADDR: %h\n", $time , rd_data_out , wr_data_in , addr_in);
            end else if (ready_out && sel_in == 1'bx) begin
                invalid_ops = invalid_ops + 1;
                $fwrite(log_file,"INVALID_OP: | Time : %0t | HRDATA : %h |
HWDATA: %h | HADDR: %h\n", $time , rd_data_out , wr_data_in , addr_in);
            end else if (ready_out && wr_enable_in == 1'bx) begin
                invalid_ops = invalid_ops + 1;
                $fwrite(log_file,"INVALID_OP: | Time : %0t | HRDATA : %h |
HWDATA: %h | HADDR: %h\n", $time , rd_data_out , wr_data_in , addr_in);
            end
            total_ops = read_ops + write_ops + invalid_ops;
        end

    final begin
        $fclose(log_file);
    end

endmodule

```

۱. راه اندازی و مقداردهی اولیه

ماژول `sdram_scoreboard` در ابتدای شبیه سازی، با استفاده از یک بلوک `initial` فایل متنی به نام `monitor_log.txt` را باز می کند. این فایل به منظور ثبت دقیق تمام عملیات های حافظه مورد استفاده قرار می گیرد. اگر این فایل به درستی باز نشود، داده های مربوط به عملیات ها ذخیره نخواهند شد. همچنین در این مرحله، سه شمارنده ی مربوط به انواع عملیات ها یعنی `write_ops` (نوشتن)، `read_ops` (خواندن) و `invalid_ops` (عملیات نامعتبر) مقداردهی اولیه شده و روی صفر تنظیم می شوند. این کار باعث می شود که ماژول بدون خطا و با دقت کامل به شمارش عملیات ها بپردازد.

۲. بررسی و ثبت عملیات های معتبر

این ماژول با استفاده از یک بلوک حساس به لبه بالارونده سیگنال کلاک (`posedge clk_in`) یا فعال شدن سیگنال ریست (`posedge rst_in`) به بررسی و تحلیل ورودی ها می پردازد. در صورت فعال بودن سیگنال ریست، شمارنده ها باید به حالت اولیه بازگردند (که در این کد به درستی بازنشانی نشده اند و بهتر است صفر شوند). در حالت عادی، ماژول به تحلیل سیگنال های ورودی زیر می پردازد:

عملیات نوشتن معتبر: اگر سیگنال انتخاب حافظه (`sel_in`) و سیگنال نوشتن (`wr_enable_in`) فعال باشند، این به معنی یک عملیات نوشتن معتبر است. در این حالت، شمارنده ی عملیات نوشتن (`write_ops`) یک واحد افزایش می یابد و جزئیات این عملیات شامل زمان شبیه سازی، داده ای که قرار است نوشته شود و آدرس آن در فایل گزارش ثبت می شود.

عملیات خواندن معتبر: اگر سیگنال انتخاب حافظه (`sel_in`) فعال باشد اما سیگنال نوشتن (`wr_enable_in`) غیرفعال باشد، این نشان دهنده ی یک عملیات خواندن معتبر است. در این حالت، شمارنده ی عملیات خواندن (`read_ops`) یک واحد افزایش یافته و اطلاعات مربوط به این عملیات نیز در فایل ثبت می شود.

۳. تشخیص و ثبت عملیات های نامعتبر

این ماژول علاوه بر عملیات های معتبر، توانایی شناسایی و ثبت عملیات های نامعتبر را نیز دارد. در موارد زیر، عملیات به عنوان نامعتبر شناخته شده و ثبت می شود:

نوشتن بدون انتخاب حافظه: اگر سیگنال آماده بودن حافظه (`ready_out`) فعال باشد اما سیگنال انتخاب حافظه (`sel_in`) غیرفعال باشد و سیگنال نوشتن (`wr_enable_in`) فعال باشد، این عملیات به عنوان نوشتن نامعتبر تلقی شده و شمارنده ی عملیات نامعتبر (`invalid_ops`) افزایش می یابد.

خواندن بدون انتخاب حافظه: اگر حافظه آماده باشد اما سیگنال انتخاب حافظه غیرفعال و سیگنال نوشتن نیز غیرفعال باشد، این عملیات به عنوان خواندن نامعتبر ثبت می شود.

وجود سیگنال های نامشخص (bx^۱): اگر سیگنال های sel_in یا wr_enable_in به مقدار نامشخص (unknown) تغییر کنند، این نیز به عنوان یک عملیات نامعتبر ثبت شده و شمارنده مربوطه افزایش می یابد.

تمامی عملیات های نامعتبر به طور دقیق با ذکر زمان شبیه سازی، داده های ورودی و خروجی و آدرس در فایل گزارش ثبت می شوند.

۴. محاسبه و به روزرسانی تعداد کل عملیات ها

پس از هر بررسی، ماژول تعداد کل عملیات ها (total_ops) را به روزرسانی می کند. این مقدار از جمع سه شمارنده ی عملیات های خواندن، نوشتن و نامعتبر محاسبه می شود. این به روزرسانی به تحلیل دقیق عملکرد سیستم کمک می کند و نشان دهنده ی تعداد کل تعاملات با حافظه در طول شبیه سازی است.

۵. مدیریت فایل و اتمام شبیه سازی

در پایان شبیه سازی، بلوک final اجرا می شود که در آن فایل monitor_log.txt بسته می شود. این کار تضمین می کند که تمام داده ها به طور کامل در فایل ذخیره شده اند و از هدررفت اطلاعات جلوگیری می شود. اگر فایل به درستی بسته نشود، ممکن است بخشی از اطلاعات ثبت نشده باقی بماند.

جمع بندی کلی عملکرد ماژول

ماژول sdram_scoreboard عملکرد سیستم را از سه جنبه ی زیر بررسی می کند:

۱. ثبت و شمارش عملیات های معتبر: عملیات های خواندن و نوشتن معتبر به صورت دقیق شمارش شده و در فایل گزارش ثبت می شوند.

۲. شناسایی و ثبت عملیات های نامعتبر: این ماژول به طور هوشمندانه هرگونه عملیات نامعتبر را تشخیص داده و ثبت می کند که به یافتن خطاها و اشکالات طراحی کمک می کند.

۳. تحلیل عملکرد سیستم: با به روزرسانی مداوم تعداد کل عملیات ها، تحلیل کلی عملکرد سیستم و شناسایی نقاط ضعف آن آسان تر می شود.

خروجی:

INVALID_OP: | Time : 105 | HRDATA : xxxxxxxx | HWDATA: 00000000 | tb_HWRITE: 0 | HADDR: 00000000
INVALID_OP: | Time : 105 | HRDATA : xxxxxxxx | HWDATA: 00000000 | tb_HWRITE: 0 | HADDR: 00000000
INVALID_OP: | Time : 115 | HRDATA : xxxxxxxx | HWDATA: 00000000 | tb_HWRITE: 0 | HADDR: 00000000
INVALID_OP: | Time : 125 | HRDATA : xxxxxxxx | HWDATA: 00000000 | tb_HWRITE: 0 | HADDR: 00000000
WRITE_OP: | Time : 135 | HWDATA : 9b75ac36 | tb_HWRITE: 1 | HADDR: 00000002
WRITE_OP: | Time : 145 | HWDATA : 9b75ac36 | tb_HWRITE: 1 | HADDR: 00000002
WRITE_OP: | Time : 155 | HWDATA : 9b75ac36 | tb_HWRITE: 1 | HADDR: 00000002
WRITE_OP: | Time : 165 | HWDATA : 9b75ac36 | tb_HWRITE: 1 | HADDR: 00000002
WRITE_OP: | Time : 175 | HWDATA : 9b75ac36 | tb_HWRITE: 1 | HADDR: 00000002
WRITE_OP: | Time : 185 | HWDATA : 9b75ac36 | tb_HWRITE: 1 | HADDR: 00000002
READ_OP: | Time : 195 | HRDATA : xxxxxxxx | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 205 | HRDATA : xxxxxxxx | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 215 | HRDATA : xxxxxxxx | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 225 | HRDATA : xxxxxxxx | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 235 | HRDATA : xxxxxxxx | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 245 | HRDATA : 9b75ac36 | tb_HWRITE: 0 | HADDR: 00000002
WRITE_OP: | Time : 255 | HWDATA : 22568d44 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 275 | HWDATA : 22568d44 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 285 | HWDATA : 22568d44 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 295 | HWDATA : 22568d44 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 305 | HWDATA : 22568d44 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 315 | HWDATA : 22568d44 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 325 | HWDATA : 22568d44 | tb_HWRITE: 1 | HADDR: 00000001
READ_OP: | Time : 335 | HRDATA : 9b75ac36 | tb_HWRITE: 0 | HADDR: 00000001
WRITE_OP: | Time : 345 | HWDATA : e8b2dad1 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 355 | HWDATA : e8b2dad1 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 365 | HWDATA : e8b2dad1 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 375 | HWDATA : e8b2dad1 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 385 | HWDATA : e8b2dad1 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 395 | HWDATA : e8b2dad1 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 405 | HWDATA : 5c8e6fb9 | tb_HWRITE: 1 | HADDR: 00000003
WRITE_OP: | Time : 415 | HWDATA : 5c8e6fb9 | tb_HWRITE: 1 | HADDR: 00000003
WRITE_OP: | Time : 425 | HWDATA : 5c8e6fb9 | tb_HWRITE: 1 | HADDR: 00000003
WRITE_OP: | Time : 435 | HWDATA : 5c8e6fb9 | tb_HWRITE: 1 | HADDR: 00000003

WRITE_OP: | Time : 445 | HWDATA : 5c8e6fb9 | tb_HWRITE: 1 | HADDR: 00000003
WRITE_OP: | Time : 455 | HWDATA : 5c8e6fb9 | tb_HWRITE: 1 | HADDR: 00000003
WRITE_OP: | Time : 465 | HWDATA : 09d79913 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 475 | HWDATA : 09d79913 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 485 | HWDATA : 09d79913 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 495 | HWDATA : 09d79913 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 505 | HWDATA : 09d79913 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 515 | HWDATA : 09d79913 | tb_HWRITE: 1 | HADDR: 00000001
READ_OP: | Time : 525 | HRDATA : 22568d44 | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 535 | HRDATA : 22568d44 | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 545 | HRDATA : 22568d44 | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 555 | HRDATA : 22568d44 | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 565 | HRDATA : 22568d44 | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 575 | HRDATA : 22568d44 | tb_HWRITE: 0 | HADDR: 00000000
WRITE_OP: | Time : 585 | HWDATA : 7163b1e2 | tb_HWRITE: 1 | HADDR: 00000002
WRITE_OP: | Time : 595 | HWDATA : 7163b1e2 | tb_HWRITE: 1 | HADDR: 00000002
WRITE_OP: | Time : 625 | HWDATA : 7163b1e2 | tb_HWRITE: 1 | HADDR: 00000002
WRITE_OP: | Time : 635 | HWDATA : 1ac90d35 | tb_HWRITE: 1 | HADDR: 00000002
WRITE_OP: | Time : 645 | HWDATA : 1ac90d35 | tb_HWRITE: 1 | HADDR: 00000002
WRITE_OP: | Time : 655 | HWDATA : 1ac90d35 | tb_HWRITE: 1 | HADDR: 00000002
WRITE_OP: | Time : 665 | HWDATA : 1ac90d35 | tb_HWRITE: 1 | HADDR: 00000002
WRITE_OP: | Time : 675 | HWDATA : 1ac90d35 | tb_HWRITE: 1 | HADDR: 00000002
READ_OP: | Time : 685 | HRDATA : 22568d44 | tb_HWRITE: 0 | HADDR: 00000003
READ_OP: | Time : 695 | HRDATA : 22568d44 | tb_HWRITE: 0 | HADDR: 00000003
READ_OP: | Time : 705 | HRDATA : 22568d44 | tb_HWRITE: 0 | HADDR: 00000003
READ_OP: | Time : 715 | HRDATA : 22568d44 | tb_HWRITE: 0 | HADDR: 00000003
READ_OP: | Time : 725 | HRDATA : 22568d44 | tb_HWRITE: 0 | HADDR: 00000003
WRITE_OP: | Time : 735 | HWDATA : 7b5399f6 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 745 | HWDATA : 7b5399f6 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 755 | HWDATA : 7b5399f6 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 765 | HWDATA : 7b5399f6 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 775 | HWDATA : 7b5399f6 | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 795 | HWDATA : 55fb75ab | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 805 | HWDATA : 55fb75ab | tb_HWRITE: 1 | HADDR: 00000001

WRITE_OP: | Time : 815 | HWDATA : 55fb75ab | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 825 | HWDATA : 55fb75ab | tb_HWRITE: 1 | HADDR: 00000001
WRITE_OP: | Time : 835 | HWDATA : 55fb75ab | tb_HWRITE: 1 | HADDR: 00000001
READ_OP: | Time : 845 | HRDATA : 7163b1e2 | tb_HWRITE: 0 | HADDR: 00000001
READ_OP: | Time : 855 | HRDATA : 7163b1e2 | tb_HWRITE: 0 | HADDR: 00000001
READ_OP: | Time : 865 | HRDATA : 7163b1e2 | tb_HWRITE: 0 | HADDR: 00000001
READ_OP: | Time : 875 | HRDATA : 7163b1e2 | tb_HWRITE: 0 | HADDR: 00000001
READ_OP: | Time : 885 | HRDATA : 7163b1e2 | tb_HWRITE: 0 | HADDR: 00000001
READ_OP: | Time : 895 | HRDATA : 7163b1e2 | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 905 | HRDATA : 55fb75ab | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 915 | HRDATA : 55fb75ab | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 925 | HRDATA : 55fb75ab | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 935 | HRDATA : 55fb75ab | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 945 | HRDATA : 55fb75ab | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 955 | HRDATA : 55fb75ab | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 965 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 975 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 985 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000000
READ_OP: | Time : 1005 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000001
READ_OP: | Time : 1015 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000001
READ_OP: | Time : 1025 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000001
READ_OP: | Time : 1035 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000001
READ_OP: | Time : 1045 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000001
WRITE_OP: | Time : 1055 | HWDATA : c18d9483 | tb_HWRITE: 1 | HADDR: 00000000
WRITE_OP: | Time : 1065 | HWDATA : c18d9483 | tb_HWRITE: 1 | HADDR: 00000000
WRITE_OP: | Time : 1075 | HWDATA : c18d9483 | tb_HWRITE: 1 | HADDR: 00000000
WRITE_OP: | Time : 1085 | HWDATA : c18d9483 | tb_HWRITE: 1 | HADDR: 00000000
WRITE_OP: | Time : 1095 | HWDATA : c18d9483 | tb_HWRITE: 1 | HADDR: 00000000
READ_OP: | Time : 1115 | HRDATA : 55fb75ab | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 1125 | HRDATA : 55fb75ab | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 1135 | HRDATA : 55fb75ab | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 1145 | HRDATA : 55fb75ab | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 1155 | HRDATA : 55fb75ab | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 1165 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000002

[illegible]

READ_OP: | Time : 1515 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 1525 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 1535 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 1545 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 1555 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 1565 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 1575 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 1585 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000002
READ_OP: | Time : 1595 | HRDATA : 09d79913 | tb_HWRITE: 0 | HADDR: 00000002

Golden model:

```
module Golden_Model_sdram(  
    input        clk_sys,  
    input        rst_sys,  
    input        write_enable,  
    input        select_line,  
    input [31:0] write_data_in,  
    input [31:0] addr_in,  
  
    output reg    ready_out,  
    output reg [31:0] read_data_out  
);  
  
parameter DATA_WIDTH = 32;  
parameter ROWS = 16384;  
parameter COLUMNS = 512;  
parameter BANKS = 4;  
  
reg [DATA_WIDTH-1:0] sdram_memory [0:BANKS-1][0:ROWS-1][0:COLUMNS-1];  
  
reg [1:0] bank_sel;  
reg [13:0] row_sel;  
reg [8:0] col_sel;  
  
typedef enum Logic [3:0] {  
    ST_IDLE = 4'b0000,  
    ST_READ_ACT = 4'b0001,  
    ST_READ_NOP1 = 4'b0010,  
    ST_READ_CAS = 4'b0011,  
};
```

```

    ST_READ_NOP2 = 4'b0100,
    ST_READ_NOP3 = 4'b0101,
    ST_WRITE_ACT = 4'b0110,
    ST_WRITE_NOP1 = 4'b0111,
    ST_WRITE_CAS = 4'b1000,
    ST_WRITE_NOP2 = 4'b1001,
    ST_WRITE_NOP3 = 4'b1010
} state_t;

state_t curr_state, next_state;

reg [31:0] data_buffer;

always @(posedge clk_sys or posedge rst_sys) begin
    if (rst_sys) begin
        curr_state <= ST_IDLE;
        ready_out <= 1'b1;
    end else begin
        curr_state <= next_state;
    end
end

always @(*) begin
    next_state = curr_state;
    ready_out = 1'b1;

    case (curr_state)
        ST_IDLE: begin
            if (select_line) begin
                if (write_enable) begin
                    next_state = ST_WRITE_ACT;
                    ready_out = 1'b0;
                end else begin
                    next_state = ST_READ_ACT;
                    ready_out = 1'b0;
                end
            end
        end
        ST_READ_ACT: begin next_state = ST_READ_NOP1; end
        ST_READ_NOP1: begin next_state = ST_READ_CAS; end
        ST_READ_CAS: begin next_state = ST_READ_NOP2; end
        ST_READ_NOP2: begin next_state = ST_READ_NOP3; end
        ST_READ_NOP3: begin next_state = ST_IDLE; end
        ST_WRITE_ACT: begin next_state = ST_WRITE_NOP1; end
    end
end

```

```

        ST_WRITE_NOP1: begin next_state = ST_WRITE_CAS; end
        ST_WRITE_CAS: begin next_state = ST_WRITE_NOP2; end
        ST_WRITE_NOP2: begin next_state = ST_WRITE_NOP3; end
        ST_WRITE_NOP3: begin next_state = ST_IDLE; end
        default: begin next_state = ST_IDLE; end

    endcase
end

always @(posedge clk_sys) begin
    if (rst_sys) begin
        row_sel <= 14'bx;
        col_sel <= 9'bx;
        bank_sel <= 2'bx;
        read_data_out <= 32'bx;
        data_buffer <= 32'bx;
    end else begin
        case (curr_state)
            ST_READ_ACT: begin
                row_sel <= addr_in[13:0];
                bank_sel <= addr_in[15:14];
            end

            ST_WRITE_ACT: begin
                row_sel <= addr_in[13:0];
                bank_sel <= addr_in[15:14];
            end

            ST_READ_CAS: begin
                col_sel <= addr_in[24:16];
                data_buffer <= sdram_memory[bank_sel][row_sel][col_sel];
            end

            ST_WRITE_CAS: begin
                col_sel <= addr_in[24:16];
                data_buffer <= write_data_in;
            end

            ST_READ_NOP2: begin
                read_data_out <= data_buffer;
            end

            ST_WRITE_NOP3: begin
                sdram_memory[bank_sel][row_sel][col_sel] <= data_buffer;
            end
        endcase
    end
end

```

```

                                default: begin
                                    end
                                endcase
                            end
                        end
                    end
                endmodule

```

توضیح:

ماژول Golden_Model_sdram یک مدل مرجع برای حافظه SDRAM است که برای شبیه‌سازی دقیق عملیات‌های خواندن و نوشتن حافظه طراحی شده است. این ماژول به‌گونه‌ای ساخته شده که بتوان عملکرد طراحی تحت تست (DUV) را با آن مقایسه کرد و از صحت عملکرد سیستم اطمینان حاصل نمود. ساختار این حافظه به‌صورت سه‌بعدی طراحی شده است و شامل چهار بانک حافظه، ۱۶۳۸۴ ردیف و ۵۱۲ ستون است که هر سلول حافظه ۳۲ بیت داده را ذخیره می‌کند. این ساختار باعث می‌شود دسترسی به داده‌ها دقیقاً مشابه رفتار واقعی حافظه SDRAM باشد.

برای مدیریت عملیات‌های خواندن و نوشتن، از یک ماشین حالت (FSM) استفاده می‌کند. این ماشین حالت به‌صورت مرحله‌ای عملیات‌ها را مدیریت می‌کند و تأخیرهای واقعی حافظه SDRAM را شبیه‌سازی می‌کند. عملیات خواندن و نوشتن شامل مراحل فعال‌سازی (Activate)، تأخیرهای نوپ (NOP)، و اجرای دستورات (CAS) است که به ترتیب اجرا می‌شوند تا رفتار حافظه به‌طور واقعی بازسازی شود. این شبیه‌سازی دقیق کمک می‌کند که تمام جنبه‌های عملکرد حافظه بررسی و تحلیل شود.

در فرآیند خواندن، ابتدا بانک، ردیف و ستون حافظه بر اساس آدرس ورودی تعیین می‌شود و سپس داده از حافظه خوانده شده و به خروجی ارسال می‌شود. در عملیات نوشتن، آدرس‌دهی انجام شده و داده ورودی در مکان مشخص شده از حافظه نوشته می‌شود. این فرآیندها با رعایت تأخیرهای استاندارد حافظه SDRAM انجام می‌شوند تا دقت شبیه‌سازی حفظ شود. همچنین سیگنال ready_out برای مدیریت وضعیت آماده بودن ماژول استفاده می‌شود. زمانی که ماژول مشغول اجرای یک عملیات است، این سیگنال غیرفعال می‌شود و پس از اتمام عملیات دوباره فعال می‌گردد. این مکانیزم از تداخل دستورات جلوگیری می‌کند و هماهنگی بین اجزای سیستم را بهبود می‌بخشد.

با استفاده از ماژول Golden_Model_sdram، می‌توان عملکرد طراحی اصلی را با یک مدل معتبر مقایسه کرده و خطاهای احتمالی را شناسایی و برطرف نمود. این ماژول نقش بسیار مهمی در تحلیل، بررسی و بهینه‌سازی عملکرد سیستم‌های دیجیتال ایفا می‌کند و باعث افزایش دقت و اطمینان از صحت عملکرد سیستم می‌شود.

خروجی:

Scenario 1: Time=176 | Mismatch: SDRAM HRDATA=xxxxxxx, HREADY=0 |tb_HWRITE: 1| Golden
HRDATA=xxxxxxx, HREADY=1

Scenario 2: Time=236 | Match: HRDATA=9b75ac36, |tb_HWRITE: 0| HREADY=1

Scenario 3: Time=327 | Mismatch: SDRAM HRDATA=9b75ac36, HREADY=0 |tb_HWRITE: 0| Golden HRDATA=xxxxxxxx, HREADY=0

Scenario 4: Time=446 | Match: HRDATA=22568d44, |tb_HWRITE: 1| HREADY=0

Scenario 5: Time=506 | Match: HRDATA=22568d44, |tb_HWRITE: 1| HREADY=0

Scenario 6: Time=566 | Match: HRDATA=22568d44, |tb_HWRITE: 0| HREADY=0

Scenario 7: Time=626 | Mismatch: SDRAM HRDATA=22568d44, HREADY=0 |tb_HWRITE: 1| Golden HRDATA=xxxxxxxx, HREADY=1

Scenario 0: Time=676 | Mismatch: SDRAM HRDATA=22568d44, HREADY=0 |tb_HWRITE: 1| Golden HRDATA=xxxxxxxx, HREADY=0

Scenario 1: Time=727 | Mismatch: SDRAM HRDATA=7163b1e2, HREADY=1 |tb_HWRITE: 0| Golden HRDATA=5c8e6fb9, HREADY=1

Scenario 2: Time=778 | Mismatch: SDRAM HRDATA=7163b1e2, HREADY=0 |tb_HWRITE: 1| Golden HRDATA=5c8e6fb9, HREADY=1

Scenario 3: Time=839 | Mismatch: SDRAM HRDATA=7163b1e2, HREADY=0 |tb_HWRITE: 1| Golden HRDATA=xxxxxxxx, HREADY=1

Scenario 4: Time=890 | Mismatch: SDRAM HRDATA=7163b1e2, HREADY=0 |tb_HWRITE: 0| Golden HRDATA=xxxxxxxx, HREADY=1

Scenario 5: Time=941 | Mismatch: SDRAM HRDATA=55fb75ab, HREADY=0 |tb_HWRITE: 0| Golden HRDATA=55fb75ab, HREADY=1

Scenario 6: Time=992 | Mismatch: SDRAM HRDATA=09d79913, HREADY=0 |tb_HWRITE: 0| Golden HRDATA=09d79913, HREADY=1

Scenario 7: Time=1053 | Mismatch: SDRAM HRDATA=55fb75ab, HREADY=1 |tb_HWRITE: 0| Golden HRDATA=xxxxxxxx, HREADY=1

Scenario 8: Time=1104 | Mismatch: SDRAM HRDATA=55fb75ab, HREADY=0 |tb_HWRITE: 1| Golden HRDATA=xxxxxxxx, HREADY=1

Scenario 9: Time=1165 | Mismatch: SDRAM HRDATA=09d79913, HREADY=1 |tb_HWRITE: 0| Golden HRDATA=xxxxxxxx, HREADY=1

Testbench:

```
module random_stimuli_generator();

    reg        tb_HCLK;
    reg        tb_HRESET;
    reg        tb_HWRITE;
    reg        tb_HSEL;
    reg [31:0] tb_HWDATA;
    reg [31:0] tb_HADDR;
```

```

wire      tb_HREADY;
wire [31:0] tb_HRDATA;

wire [31:0] golden_HRDATA;
wire golden_HREADY;
wire [15:0] error_count;

integer    file;
integer    i;
integer    seed;

wire [31:0] total_ops;
wire [31:0] read_ops;
wire [31:0] write_ops;
wire [31:0] invalid_ops;

sdram_top DUT (
    .in_HCLK(tb_HCLK),
    .in_HRESET(tb_HRESET),
    .in_HWRITE(tb_HWRITE),
    .in_HSEL(tb_HSEL),
    .in_HWDATA(tb_HWDATA),
    .in_HADDR(tb_HADDR),
    .out_HREADY(tb_HREADY),
    .out_HRDATA(tb_HRDATA)
);

Golden_Model_sdram GoldenModel(
    .clk_sys(tb_HCLK),
    .rst_sys(tb_HRESET),
    .write_enable(tb_HWRITE),
    .select_line(tb_HSEL),
    .write_data_in(tb_HWDATA),
    .addr_in(tb_HADDR),
    .ready_out(golden_HREADY),
    .read_data_out(golden_HRDATA)
);

Checker checker_inst (
    .clk(tb_HCLK),
    .reset(tb_HRESET),
    .golden_HRDATA(golden_HRDATA),
    .tb_HRDATA(tb_HRDATA),
    .golden_HREADY(golden_HREADY),
    .tb_HREADY(tb_HREADY),

```

```

        .address(tb_HADDR),
        .error_count(error_count)
    );

    sdram_scoreboard u_sdram_scoreboard (
        .clk_in      (tb_HCLK),
        .rst_in      (tb_HRESET),
        .wr_enable_in (tb_HWRITE),
        .sel_in      (tb_HSEL),
        .addr_in     (tb_HADDR),
        .wr_data_in  (tb_HWDATA),
        .ready_out   (tb_HREADY),
        .rd_data_out (tb_HRDATA),

        .total_ops   (total_ops),
        .read_ops    (read_ops),
        .write_ops    (write_ops),
        .invalid_ops (invalid_ops)
    );

    // Clock generation
    initial begin
        tb_HCLK = 0;
        forever #5 tb_HCLK = ~tb_HCLK;
    end

    // Stimulus
    initial begin
        seed=10;
        tb_HRESET = 1;
        tb_HWRITE = 0;
        tb_HSEL = 0;
        tb_HWDATA = 32'h0;
        tb_HADDR = 32'h0;

        // Reset period
        #100;
        tb_HRESET = 0;
        #20;

        file = $fopen("deterministic_output5.txt", "w");

        // Scenario 1: Simple Write Operation with random data
        @(posedge tb_HCLK);
        tb_HSEL = 1;

```

```

tb_HWRITE = 1;
tb_HADDR = $random/32'h40000000;
tb_HWDATA = $random(seed);

#50;
#1;
$fwrite(file, "Scenario 1: Time: %0t | tb_HWRITE: %b | tb_HWDATA: %h |
tb_HREADY: %b | tb_HRDATA: %h\n", $time, tb_HWRITE, tb_HWDATA, tb_HREADY,
tb_HRDATA);

// Scenario 2: Simple Read Operation with random address
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 0;
tb_HADDR = tb_HADDR;

#50;
#1;
$fwrite(file, "Scenario 2: Time: %0t | tb_HWRITE: %b | tb_HREADY: %b |
tb_HRDATA: %h\n", $time, tb_HWRITE, tb_HREADY, tb_HRDATA);

// Scenario 3: Write, Reset, and Read Operation with random data
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = $random;
tb_HADDR = $random/32'h40000000;

#20;
tb_HRESET = 1;
#10;
tb_HRESET = 0;
#50;
tb_HSEL = 1;
tb_HWRITE = 0;
tb_HADDR = tb_HADDR;

#1;
$fwrite(file, "Scenario 3: Time: %0t | tb_HWRITE: %b | tb_HWDATA: %h |
tb_HREADY: %b | tb_HRDATA: %h\n", $time, tb_HWRITE, tb_HWDATA, tb_HREADY,
tb_HRDATA);

// Scenario 4: Multiple Write Operations with random data and addresses
@(posedge tb_HCLK);
tb_HSEL = 1;

```

```

tb_HWRITE = 1;
tb_HWDATA = $random;
tb_HADDR = $random/32'h40000000;

#50;
#1;
$fwrite(file, "Scenario 4: Time: %0t | tb_HWRITE: %b | tb_HWDATA: %h |
tb_HREADY: %b | tb_HRDATA: %h\n", $time, tb_HWRITE, tb_HWDATA, tb_HREADY,
tb_HRDATA);

@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = $random;
tb_HADDR = $random/32'h40000000;

#50;
#1;
$fwrite(file, "Scenario 4: Time: %0t | tb_HWRITE: %b | tb_HWDATA: %h |
tb_HREADY: %b | tb_HRDATA: %h\n", $time, tb_HWRITE, tb_HWDATA, tb_HREADY,
tb_HRDATA);

// Scenario 5: Bank Boundary Test with random address
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = $random;
tb_HADDR = $random/32'h40000000;

#50;
#1;
$fwrite(file, "Scenario 5: Time: %0t | tb_HWRITE: %b | tb_HREADY: %b |
tb_HRDATA: %h\n", $time, tb_HWRITE, tb_HREADY, tb_HRDATA);

// Scenario 6: Sequential Read After Write in Different Banks
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 0;
tb_HADDR = $random/32'h40000000;

#50;
#1;
$fwrite(file, "Scenario 6: Time: %0t | tb_HWRITE: %b | tb_HREADY: %b |
tb_HRDATA: %h\n", $time, tb_HWRITE, tb_HREADY, tb_HRDATA);

```

```

// Scenario 7: Reset During Operation with random data
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = $random;
tb_HADDR = $random/32'h40000000;

#30;
tb_HRESET = 1;
#20;
tb_HRESET = 0;
#10;
$fwrite(file, "Scenario 7: Time: %0t | tb_HWRITE: %b | tb_HREADY: %b |
Reset Triggered\n", $time, tb_HWRITE, tb_HREADY);

// Loop for additional random operations
for (i = 0; i < 10; i = i + 1) begin
    tb_HADDR = $random/32'h40000000;
    tb_HWDATA = $random;
    tb_HWRITE = $random % 2;
    tb_HSEL = 1;

    if ($random % 2 == 0) begin
        tb_HRESET = 1;
        #10;
        tb_HRESET = 0;
    end

    #10;
    $fwrite(file, "Scenario %0d: Time: %0t | tb_HADDR: %h | tb_HWDATA: %h
| tb_HWRITE: %b | tb_HREADY: %b | tb_HRDATA: %h\n",
        i, $time, tb_HADDR, tb_HWDATA, tb_HWRITE, tb_HREADY,
tb_HRDATA);

    #50;
end

// Close file
$fclose(file);

end
endmodule

```

توضیح:

محرک‌ها و سناریوهای تست

بخش اصلی ماژول، تولید محرک‌های تصادفی و اجرای سناریوهای مختلف تست است. نتایج هر سناریو در فایل ذخیره می‌شود.

سناریو ۱: نوشتن ساده با داده تصادفی؛ یک عملیات نوشتن با آدرس و داده تصادفی انجام می‌شود. نتیجه این عملیات در فایل ثبت می‌شود.

سناریو ۲: خواندن ساده با آدرس تصادفی؛ یک عملیات خواندن با آدرس تصادفی انجام می‌شود. نتیجه عملیات خواندن ثبت می‌شود.

سناریو ۳: نوشتن، ریست و خواندن؛ ابتدا یک عملیات نوشتن انجام می‌شود. سپس سیستم ریست می‌شود و بعد از آن یک عملیات خواندن انجام می‌شود. این سناریو برای بررسی تأثیر ریست روی داده‌ها طراحی شده است.

سناریو ۴: چندین عملیات نوشتن پشت سر هم؛ چند عملیات نوشتن با داده و آدرس تصادفی اجرا می‌شود. نتایج در فایل ثبت می‌شود.

سناریو ۵: تست مرز بانک‌های حافظه؛ یک عملیات نوشتن روی مرزهای بانک حافظه انجام می‌شود. هدف این سناریو بررسی دسترسی صحیح به بانک‌های حافظه است.

سناریو ۶: خواندن متوالی پس از نوشتن در بانک‌های مختلف؛ چند عملیات نوشتن و بلافاصله خواندن در بانک‌های مختلف انجام می‌شود. برای بررسی صحت دسترسی به حافظه در بانک‌های مختلف است.

سناریو ۷: ریست حین عملیات؛ یک عملیات نوشتن شروع می‌شود، سپس در میانه آن ریست اعمال می‌شود. هدف این سناریو بررسی مقاومت سیستم در برابر ریست ناگهانی است.

سناریو ۸: تولید محرک‌های تصادفی متوالی؛ یک حلقه ۱۰ بار عملیات‌های تصادفی خواندن و نوشتن را انجام می‌دهد. به‌صورت تصادفی ریست اعمال می‌شود و تمامی نتایج ثبت می‌شود.

نتیجه:

Scenario 1: Time: 176 | tb_HWRITE: 1 | tb_HWDATA: 9b75ac36 | tb_HREADY: 0 | tb_HRDATA: xxxxxxxx

Scenario 2: Time: 236 | tb_HWRITE: 0 | tb_HREADY: 1 | tb_HRDATA: 9b75ac36

Scenario 3: Time: 327 | tb_HWRITE: 0 | tb_HWDATA: 22568d44 | tb_HREADY: 0 | tb_HRDATA: 9b75ac36

Scenario 4: Time: 386 | tb_HWRITE: 1 | tb_HWDATA: e8b2dad1 | tb_HREADY: 0 | tb_HRDATA: 22568d44

Scenario 4: Time: 446 | tb_HWRITE: 1 | tb_HWDATA: 5c8e6fb9 | tb_HREADY: 0 | tb_HRDATA: 22568d44

Scenario 5: Time: 506 | tb_HWRITE: 1 | tb_HREADY: 0 | tb_HRDATA: 22568d44

Scenario 6: Time: 566 | tb_HWRITE: 0 | tb_HREADY: 0 | tb_HRDATA: 22568d44

Scenario 7: Time: 626 | tb_HWRITE: 1 | tb_HREADY: 0 | Reset Triggered

Scenario 0: Time: 676 | tb_HADDR: 00000002 | tb_HWDATA: 1ac90d35 | tb_HWRITE: 1 | tb_HREADY: 0 | tb_HRDATA: 22568d44

Scenario 1: Time: 727 | tb_HADDR: 00000003 | tb_HWDATA: 2a78bf54 | tb_HWRITE: 0 | tb_HREADY: 1 | tb_HRDATA: 7163b1e2

Scenario 2: Time: 778 | tb_HADDR: 00000001 | tb_HWDATA: 7b5399f6 | tb_HWRITE: 1 | tb_HREADY: 0 | tb_HRDATA: 7163b1e2

Scenario 3: Time: 839 | tb_HADDR: 00000001 | tb_HWDATA: 55fb75ab | tb_HWRITE: 1 | tb_HREADY: 0 | tb_HRDATA: 7163b1e2

Scenario 4: Time: 890 | tb_HADDR: 00000001 | tb_HWDATA: a095f241 | tb_HWRITE: 0 | tb_HREADY: 0 | tb_HRDATA: 7163b1e2

Scenario 5: Time: 941 | tb_HADDR: 00000000 | tb_HWDATA: 4c686d98 | tb_HWRITE: 0 | tb_HREADY: 0 | tb_HRDATA: 55fb75ab

Scenario 6: Time: 992 | tb_HADDR: 00000000 | tb_HWDATA: 6333fdc6 | tb_HWRITE: 0 | tb_HREADY: 0 | tb_HRDATA: 09d79913

Scenario 7: Time: 1053 | tb_HADDR: 00000001 | tb_HWDATA: 10518120 | tb_HWRITE: 0 | tb_HREADY: 1 | tb_HRDATA: 55fb75ab

Scenario 8: Time: 1104 | tb_HADDR: 00000000 | tb_HWDATA: c18d9483 | tb_HWRITE: 1 | tb_HREADY: 0 | tb_HRDATA: 55fb75ab

Scenario 9: Time: 1165 | tb_HADDR: 00000002 | tb_HWDATA: 3bc49777 | tb_HWRITE: 0 | tb_HREADY: 1 | tb_HRDATA: 09d79913

خروجی wave:



